

upfbench — Runbook

Setup, dependencies, and how to run the test suites

coRAN Labs

June 2026

Contents

0. What the framework does	1
1. Prerequisites (in order)	2
2. Install the framework	3
3. Configure the campaign	3
4. Run it	4
5. What each suite does (and what it produces)	4
6. Outputs	5
7. Operational notes (read these)	5
8. Troubleshooting	5
9. What's left / roadmap	6

A step-by-step, dependency-complete guide to running upfbench against an SD-Core BESS-UPF. Everything here was validated end-to-end on this VM (Ubuntu 22.04, single virtio NIC, SD-Core BESS-UPF in **af_packet** mode).

TL;DR: deploy the UPF → install host deps → build pfcpsim → (optional) install LaTeX → edit configs/sdcore-bess.yaml → python3 -m upfbench.cli run --config ... --suite all.

0. What the framework does

upfbench points at a 5G UPF, runs a **test suite**, and produces a standards-aligned report. Three suites:

Suite	What it measures	Needs
performance	single-tunnel throughput (NDR/PDR), latency/jitter, burst, multi-flow	pybess (rules) + tcpreplay (GTP-U)
load	many UEs: session capacity, per-UE throughput, latency-under-load	pfcpsim (N sessions) + tcpreplay (per-TEID GTP-U)

Suite	What it measures	Needs
pfcp	N4/PFCP conformance per 3GPP TS 29.244 (pass/fail matrix)	pfcp sim only (no traffic)

It connects to an **already-running** UPF (it does not deploy the UPF).

1. Prerequisites (in order)

1.1 — The UPF must be deployed and healthy

Deploy SD-Core + BESS-UPF in **af_packet** mode via Aether OnRamp (see ~/SD-CORE_AETHER_ONRAMP_DEPLOY.md). Confirm it's up:

```
export KUBECONFIG=$HOME/.kube/config
export PATH=$PATH:/var/lib/rancher/rke2/bin
kubectl get pod upf-0 -n aether-5gc          # expect: upf-0 5/5 Running
kubectl exec -n aether-5gc upf-0 -c bessd -- bessctl show port | grep -E "accessFast|coreFast"
```

The host must reach the pod's access interface (the af_packet N3 ingress):

```
ping -c2 -I access 192.168.252.3           # pod access IP; expect 0% loss
```

1.2 — kubectl access

The adapter and pfcp sim control shell into the pod via kubectl. Make sure: - kubectl is on PATH (RKE2 puts it in /var/lib/rancher/rke2/bin), - KUBECONFIG points at a config that can reach the cluster (~/kube/config).

1.3 — Passwordless sudo (for the traffic suites)

The traffic generator runs `sudo tcpreplay` and `sudo taskset`. The user running `upfbench` needs passwordless sudo (the pfcp suite alone does NOT need sudo).

1.4 — Host packages

```
sudo apt-get update
sudo apt-get install -y python3 python3-pip tcpreplay
```

1.5 — Python dependencies

```
pip3 install --user "PyYAML>=6.0" "Jinja2>=3.1" scapy
```

- PyYAML, Jinja2 — config + report rendering (declared in pyproject.toml).
- scapy — the generator crafts GTP-U packets into a pcap (Suites 1 & 2).

1.6 — Go + build pfcpsim (Suites 2 & 3 only)

pfcpsim is vendored as Go source in `third_party/pfcpsim/`; build the two binaries. It needs **Go ≥ 1.25** (Ubuntu's apt Go is too old — install from the tarball):

```
# install a current Go
cd /tmp && curl -sLO https://go.dev/dl/go1.26.4.linux-amd64.tar.gz
sudo rm -rf /usr/local/go && sudo tar -C /usr/local -xzf go1.26.4.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin

# build pfcpsim + pfcpcctl
cd ~/Shiva/upf-benchmark-framework/third_party/pfcpsim
GOFLAGS=-mod=mod GOT00LCHAIN=local go build -o pfcpsim ./cmd/pfcpsim
GOFLAGS=-mod=mod GOT00LCHAIN=local go build -o pfcpcctl ./cmd/pfcpcctl
ls -l pfcpsim pfcpcctl      # two binaries; gitignored (build locally per host)
```

1.7 — LaTeX (optional, for PDF reports)

Without it, reports are written as `.tex` (still complete); with it you get `.pdf`.

```
sudo apt-get install -y texlive-latex-base texlive-latex-recommended \
    texlive-latex-extra texlive-fonts-recommended
```

2. Install the framework

Pure-Python, no build needed. Run it as a module (always works):

```
cd ~/Shiva/upf-benchmark-framework
export KUBECONFIG=$HOME/.kube/config
export PATH=$PATH:/var/lib/rancher/rke2/bin:$HOME/.local/bin:/usr/local/go/bin
```

```
python3 -m upfbench.cli list      # sanity check: lists suites + test cases
```

(Optional: `pip install -e .` gives an `upfbench` command, but needs `setuptools ≥ 64`.)

3. Configure the campaign

Edit `configs/sdcore-bess.yaml`. Key fields:

```
campaign: UPF-BM-2026-001      # output dir name under campaigns/
sut: { cpu: ..., nic: ..., kernel: ... } # static facts; adapter overlays live image/mode/ports

upf:
  adapter: sdcore_bess
  n3_iface: access              # BESS access port (N3 ingress)
  n6_iface: core                # BESS core port (N6 egress)
  namespace: aether-5gc        # k8s location of the UPF pod
  pod: upf-0
  bessd_container: bessd
  gen_iface: access             # host interface the generator sends from
  n3_remote_ip: "192.168.252.3" # UPF N3 IP (MAC auto-read from the pod)
  gen_cpu_affinity: auto        # pin generator OFF bessd's worker core (~3x effect)
```

```

suite: performance                # default suite; override with --suite

performance:                      # Suite 1 knobs
  control: pybess
  generator: tcpreplay
  frame_sizes: [128, 512, 1024, 1518]
  trial_duration_s: 5             # raise (e.g. 30-60) for publication-grade stability
  max_rate_mpps: 0.12            # NDR/PDR search ceiling
  ...
load:                             # Suite 2 knobs
  control: pfcpsim
  generator: tcpreplay
  ue_counts: [10, 100, 1000, 5000] # LT-01 capacity ramp
  lt02_ue_count: 100              # LT-02 throughput UE count
  lt03_ue_counts: [1, 10, 100]    # LT-03 latency-vs-load points
  ...
pfcf:                             # Suite 3 (no extra knobs needed)
  procedures: [association, establish, modify, delete, error_handling]

# baseline: campaigns/UPF-BM-2026-000/results.json # uncomment for a comparison section

```

4. Run it

```

cd ~/Shiva/upf-benchmark-framework
export KUBECONFIG=$HOME/.kube/config
export PATH=$PATH:/var/lib/rancher/rke2/bin:$HOME/.local/bin

# list suites + test cases
python3 -m upfbench.cli list

# run one suite
python3 -m upfbench.cli run --config configs/sdcore-bess.yaml --suite performance
python3 -m upfbench.cli run --config configs/sdcore-bess.yaml --suite load
python3 -m upfbench.cli run --config configs/sdcore-bess.yaml --suite pfcf

# run everything in one campaign
python3 -m upfbench.cli run --config configs/sdcore-bess.yaml --suite all

# interactive menu
python3 -m upfbench.cli

```

Approx. run times on this VM (default knobs): performance ~3-4 min, load ~2-3 min, pfcf ~30 s, all ~6-7 min.

5. What each suite does (and what it produces)

performance — installs a wildcard “forward-all” rule via pybess, then tcpreplay blasts GTP-U from the host into the UPF’s access socket: - **TC-01** NDR/PDR per frame size (binary search), **TC-03** in-pipeline latency, **TC-04** burst (drops/drain), **TC-08** multi-flow. **TC-02** bidirectional is skipped (needs a downlink path — see §8).

load — pfcpsim installs N real per-UE PFCP sessions; tcpreplay drives GTP-U on the

matching per-UE TEIDs: - **LT-01** session capacity ramp, **LT-02** aggregate + per-UE forwarding verification, **LT-03** latency vs UE count.

pfcpsim — pfcpsim drives each N4 procedure and asserts the response: - **CF-01** association, **CF-02** establish, **CF-03** modify, **CF-04** delete, **CF-05** unknown-SEID rejection. Output is a pass/fail matrix.

6. Outputs

```
campaigns/<campaign-id>/
results.json          # machine-readable: KPIs, per-test tables, captured commands
report-performance.pdf # one report per suite that ran (.tex if no LaTeX)
report-load.pdf
report-pfcpsim.pdf
raw/                  # pybess scripts, pfcpsim server log, etc.
```

7. Operational notes (read these)

- **Start from a clean UPF for trustworthy numbers.** Heavy session churn degrades the pfcpsim-agent→bessd datapath over a long session. If results look wrong (e.g. 0 forwarded), reset the pod:

```
kubectl delete pod upf-0 -n aether-5gc
kubectl wait --for=condition=ready pod/upf-0 -n aether-5gc --timeout=120s
```
- **datapath down auto-recovery:** the pfcpsim control detects this and restarts the pfcpsim-agent + re-associates automatically — no action needed mid-run.
- **CPU affinity matters (~3×):** gen_cpu_affinity: auto keeps the software generator off bessd's worker core. Leave it on auto.
- **af_packet is non-monotonic:** the no-drop rate (TC-01) can exceed the heavy-overload rate (TC-04) because the kernel socket livelocks under saturation. Numbers are also noisy at these low rates — **raise trial_duration_s** for stable, publication-grade results.
- **TRex vs tcpreplay:** this VM has no DPDK NIC, so we use tcpreplay. On real DPDK hardware, set generator: trex (the plugin is wired) for higher rates + per-flow stats.

8. Troubleshooting

Symptom	Cause / fix
kubectl exec ... command not found	KUBECONFIG/PATH not set (see §2 exports)

Symptom	Cause / fix
Generator sends but 0 forwarded	UPF datapath degraded — reset the pod (\$7)
tcpreplay: command not found / permission	install tcpreplay; ensure passwordless sudo
pfcp sim could not connect / datapath down	auto-recovered; if persistent, reset the pod
pfcp sim build fails on Go version	install Go ≥ 1.25 from the tarball (\$1.6)
Report stays .tex (no PDF)	LaTeX not installed (\$1.7) — .tex is still complete
TC-01 numbers vary run-to-run	af_packet low-rate noise — raise trial_duration_s

9. What's left / roadmap

Working today (validated end-to-end): all three suites against SD-Core af_packet — performance (TC-01/03/04/08), load (LT-01/02/03 incl. per-UE forwarding verification), pfcp conformance (CF-01..05), plus per-suite PDF reports with baseline comparison.

Deferred / future: 1. **TC-02 bidirectional** — needs a real downlink GTP-U path (comes naturally with TRex, or a DL session + GTP-encap setup on this VM). Currently skipped. 2. **True per-UE fairness (LT-02)** — current per-UE is the fair-share average plus an individual forwarding check; true under-load per-UE share needs BESS FlowMeasure per-flow counters (blocked on a multi-worker “leader flip” restriction). 3. **Worker auto-tune (TC-05)** — auto-pick the best bessd worker count/core pinning; limited on this VM (chart caps at 2 workers, contended cores) — more valuable on dedicated isolated-core hardware. 4. **More UPF adapters** — OAI-UPF / Open5GS / free5GC / eUPF (drop a new adapters/<x>.py; the suites don't change). 5. **TRex generator** — implement traffic/trex.py for high-rate + per-flow stats on DPDK hardware.

Validated: 2026-06-14, SD-Core BESS-UPF af_packet (upf-bess rel-2.4.3), single-NIC VM.